

Find the Error

Each of the following programs or program segments has errors. Find as many as you can.

- ```

65. fstream file(ios::in | ios::out);
 file.open("info.dat");
 if (!file)
 {
 cout << "Could not open file.\n";
 }

66. ofstream file;
 file.open("info.dat", ios::in);
 if (file)
 {
 cout << "Could not open file.\n";
 }

67. fstream file("info.dat");
 if (!file)
 {
 cout << "Could not open file.\n";
 }

68. fstream dataFile("info.dat", ios::in | ios::binary);
 int x = 5;
 dataFile << x;

69. fstream dataFile("info.dat", ios::in);
 char stuff[81];
 dataFile.get(stuff);

70. fstream dataFile("info.dat", ios::in);
 char stuff[81] = "abcdefghijklmnopqrstuvwxyz";
 dataFile.put(stuff);

71. fstream dataFile("info.dat", ios::out);
 struct Date
 {
 int month;
 int day;
 int year;
 };
 Date dt = { 4, 2, 98 };
 dataFile.write(&dt, sizeof(int));

72. fstream inFile("info.dat", ios::in);
 int x;
 inFile.seekp(5);
 inFile >> x;

```

**Programming Challenges****1. File Head Program**

Write a program that asks the user for the name of a file. The program should display the first 10 lines of the file on the screen (the “head” of the file). If the file has fewer

than 10 lines, the entire file should be displayed, with a message indicating the entire file has been displayed.



**NOTE:** Using an editor, you should create a simple text file that can be used to test this program.

## 2. File Display Program

Write a program that asks the user for the name of a file. The program should display the contents of the file on the screen. If the file's contents won't fit on a single screen, the program should display 24 lines of output at a time, and then pause. Each time the program pauses, it should wait for the user to strike a key before the next 24 lines are displayed.



**NOTE:** Using an editor, you should create a simple text file that can be used to test this program.

## 3. Punch Line

Write a program that reads and prints a joke and its punch line from two different files. The first file contains a joke, but not its punch line. The second file has the punch line as its last line, preceded by "garbage." The `main` function of your program should open the two files and then call two functions, passing each one the file it needs. The first function should read and display each line in the file it is passed (the joke file). The second function should display only the last line of the file it is passed (the punch line file). It should find this line by seeking to the end of the file and then backing up to the beginning of the last line. Data to test your program can be found in the `joke.txt` and `punchline.txt` files.

## 4. Tail Program

Write a program that asks the user for the name of a file. The program should display the last 10 lines of the file on the screen (the "tail" of the file). If the file has fewer than 10 lines, the entire file should be displayed, with a message indicating the entire file has been displayed.



**NOTE:** Using an editor, you should create a simple text file that can be used to test this program.

## 5. Line Numbers

(This assignment could be done as a modification of the program in Programming Challenge 2.) Write a program that asks the user for the name of a file. The program should display the contents of the file on the screen. Each line of screen output should be preceded with a line number, followed by a colon. The line numbering should start at 1. Here is an example:

```
1:George Rolland
2:127 Academy Street
3:Brasstown, NC 28706
```

If the file's contents won't fit on a single screen, the program should display 24 lines of output at a time, and then pause. Each time the program pauses, it should wait for the user to strike a key before the next 24 lines are displayed.



**NOTE:** Using an editor, you should create a simple text file that can be used to test this program.

## 6. String Search

Write a program that asks the user for a file name and a string to search for. The program should search the file for every occurrence of a specified string. When the string is found, the line that contains it should be displayed. After all the occurrences have been located, the program should report the number of times the string appeared in the file.



**NOTE:** Using an editor, you should create a simple text file that can be used to test this program.

## 7. Sentence Filter

Write a program that asks the user for two file names. The first file will be opened for input and the second file will be opened for output. (It will be assumed that the first file contains sentences that end with a period.) The program will read the contents of the first file and change all the letters to lowercase except the first letter of each sentence, which should be made uppercase. The revised contents should be stored in the second file.



**NOTE:** Using an editor, you should create a simple text file that can be used to test this program.

## 8. Array/File Functions

Write a function named `arrayToFile`. The function should accept three arguments: the name of a file, a pointer to an `int` array, and the size of the array. The function should open the specified file in binary mode, write the contents of the array to the file, and then close the file.

Write another function named `fileToArray`. This function should accept three arguments: the name of a file, a pointer to an `int` array, and the size of the array. The function should open the specified file in binary mode, read its contents into the array, and then close the file.

Write a complete program that demonstrates these functions by using the `arrayToFile` function to write an array to a file, and then using the `fileToArray` function to read the data from the same file. After the data are read from the file into the array, display the array's contents on the screen.



VideoNote  
Solving the File  
Encryption Filter  
Problem

### 9. File Encryption Filter

File encryption is the science of writing the contents of a file in a secret code. Your encryption program should work like a filter, reading the contents of one file, modifying the data into a code, and then writing the coded contents out to a second file. The second file will be a version of the first file, but written in a secret code.

Although there are complex encryption techniques, you should come up with a simple one of your own. For example, you could read the first file one character at a time, and add 10 to the ASCII code of each character before it is written to the second file.

### 10. File Decryption Filter

Write a program that decrypts the file produced by the program in Programming Challenge 9. The decryption program should read the contents of the coded file, restore the data to its original state, and write it to another file.

### 11. Corporate Sales Data Output

Write a program that uses a structure to store the following data on a company division:

Division Name (such as East, West, North, or South)  
Quarter (1, 2, 3, or 4)  
Quarterly Sales

The user should be asked for the four quarters' sales figures for the East, West, North, and South divisions. The data for each quarter for each division should be written to a file.

*Input Validation: Do not accept negative numbers for any sales figures.*

### 12. Corporate Sales Data Input

Write a program that reads the data in the file created by the program in Programming Challenge 11. The program should calculate and display the following figures:

- Total corporate sales for each quarter
- Total yearly sales for each division
- Total yearly corporate sales
- Average quarterly sales for the divisions
- The highest and lowest quarters for the corporation

### 13. Inventory Program

Write a program that uses a structure to store the following inventory data in a file:

Item Description  
Quantity on Hand  
Wholesale Cost  
Retail Cost  
Date Added to Inventory

The program should have a menu that allows the user to perform the following tasks:

- Add new records to the file.
- Display any record in the file.
- Change any record in the file.

*Input Validation: The program should not accept quantities, or wholesale or retail costs, less than 0. The program should not accept dates that the programmer determines are unreasonable.*

#### 14. Inventory Screen Report

Write a program that reads the data in the file created by the program in Programming Challenge 13. The program should calculate and display the following data:

- The total wholesale value of the inventory
- The total retail value of the inventory
- The total quantity of all items in the inventory

#### 15. Average Number of Words

If you have downloaded this book's source code from the companion Web site, you will find a file named `text.txt` in the Chapter 12 folder. (The companion Web site is at [www.pearsonhighered.com/gaddis](http://www.pearsonhighered.com/gaddis).) The text that is in the file is stored as one sentence per line. Write a program that reads the file's contents and calculates the average number of words per sentence.

### Group Project

#### 16. Customer Accounts

This program should be designed and written by a team of students. Here are some suggestions:

- One student should design function `main`, which will call other program functions. The remainder of the functions will be designed by other members of the team.
- The requirements of the program should be analyzed so each student is given about the same workload.

Write a program that uses a structure to store the following data about a customer account:

Name  
Address  
City, State, and ZIP  
Telephone Number  
Account Balance  
Date of Last Payment

The structure should be used to store customer account records in a file. The program should have a menu that lets the user perform the following operations:

- Enter new records into the file.
- Search for a particular customer's record and display it.
- Search for a particular customer's record and delete it.
- Search for a particular customer's record and change it.
- Display the contents of the entire file.

*Input Validation: When the data for a new account is entered, be sure the user enters data for all the fields. No negative account balances should be entered.*